

VULNERABILITY ASSESSMENT REPORT

OWASP Juice Shop

Future InternsCyberSecurityInternship— Task 1

Yousef Ahmed Fawzy

CIN ID: FIT/JUN26/CS9482

Cyber Security Track (CS) | 30 June 2026

Passive Read-Only Assessment | Ethical Scope | No Exploitation

1. Executive Summary

This report presents the findings of a passive vulnerability assessment conducted against OWASP Juice Shop, a deliberately vulnerable web application hosted locally via Docker on Kali Linux. The assessment was performed as Task 1 of the Future Interns Cyber Security Internship and strictly follows ethical guidelines — read-only, passive analysis only, with no exploitation, brute force, or active attacks of any kind.

A total of **6 vulnerabilities** were identified: **4 Medium** risk and **2 Low** risk. The most significant findings involve missing security headers and a misconfigured CORS policy. All findings include practical remediation steps written in business-friendly language.

Risk Level	Count	Status
High	0	None Identified
Medium	4	Requires Attention
Low	2	Monitor & Review
Total Findings	6	—

Target Application — OWASP Juice Shop

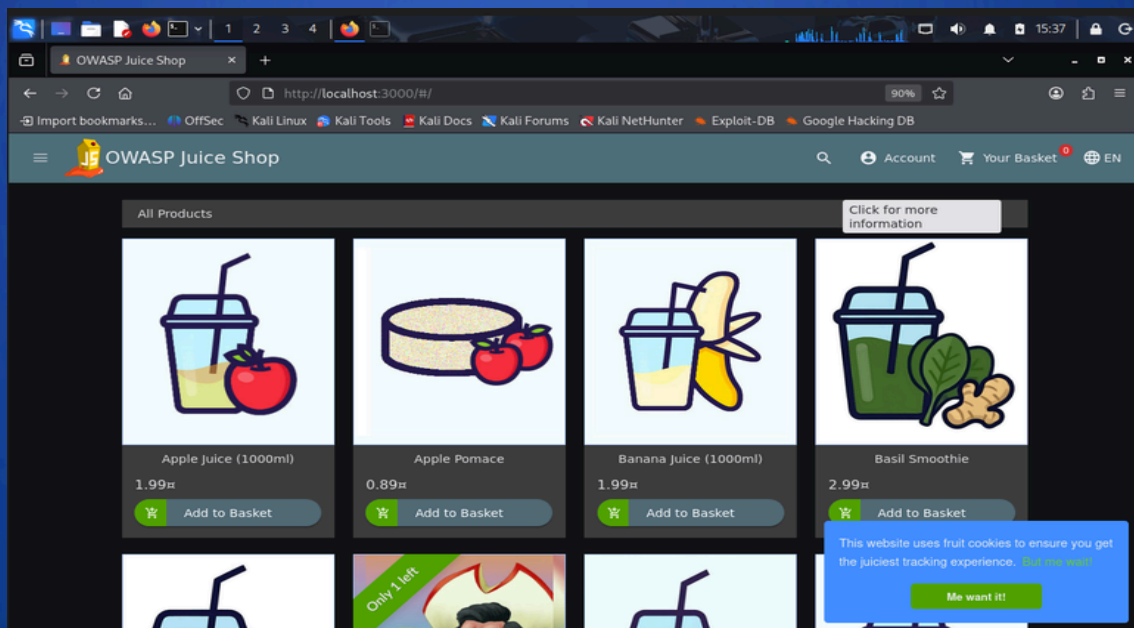


Figure 1: OWASP Juice Shop running on http://localhost:3000

2. Scope & Methodology

Item	Details
Application	OWASP Juice Shop (deliberately vulnerable demo app)
URL	http://localhost:3000
Hosting	Docker container on Kali Linux (local environment)
Type	Demo/training web application — legal to test
Assessment Date	30 June 2026
Assessor	Yousef Ahmed Fawzy FIT/JUN26/CS9482

Allowed: Public-facing pages, passive scanning, header checks, configuration analysis

Not Allowed: Login bypass, exploitation, brute force, DoS, active scanning

Tool	Version	Purpose
Nmap	7.95	Port scanning and service/version detection
Browser DevTools	Firefox	HTTP response header analysis
OWASP ZAP	2.17.0	Passive vulnerability scanning via proxy
Canva	Web	Professional report design

3.1 Network Scanning — Nmap

Nmap was used to identify open ports and running services. Port 3000 was found open running an HTTP service. The service fingerprint captured HTTP response headers directly, revealing the CORS wildcard and X-Recruiting information disclosure header from the scan output.

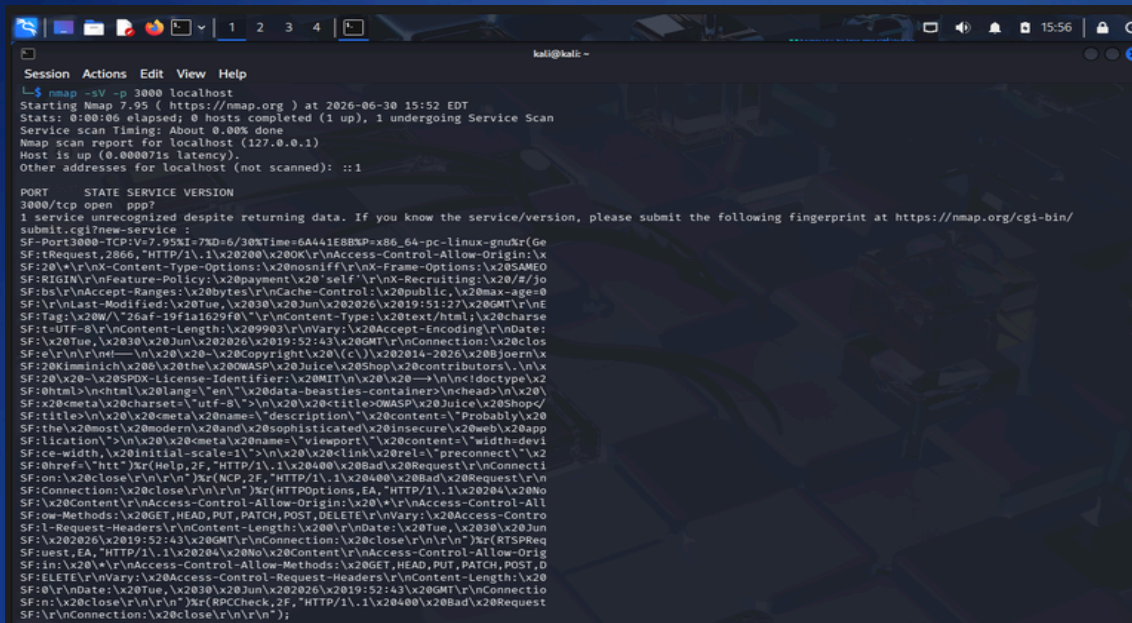


Figure 2: Nmap scan — port 3000 open, HTTP fingerprint reveals CORS wildcard and X-Recruiting headers

3.2 HTTP Header Analysis — Browser DevTools

Firefox DevTools captured all HTTP requests during page load. Response Headers revealed both present and critically missing security headers.

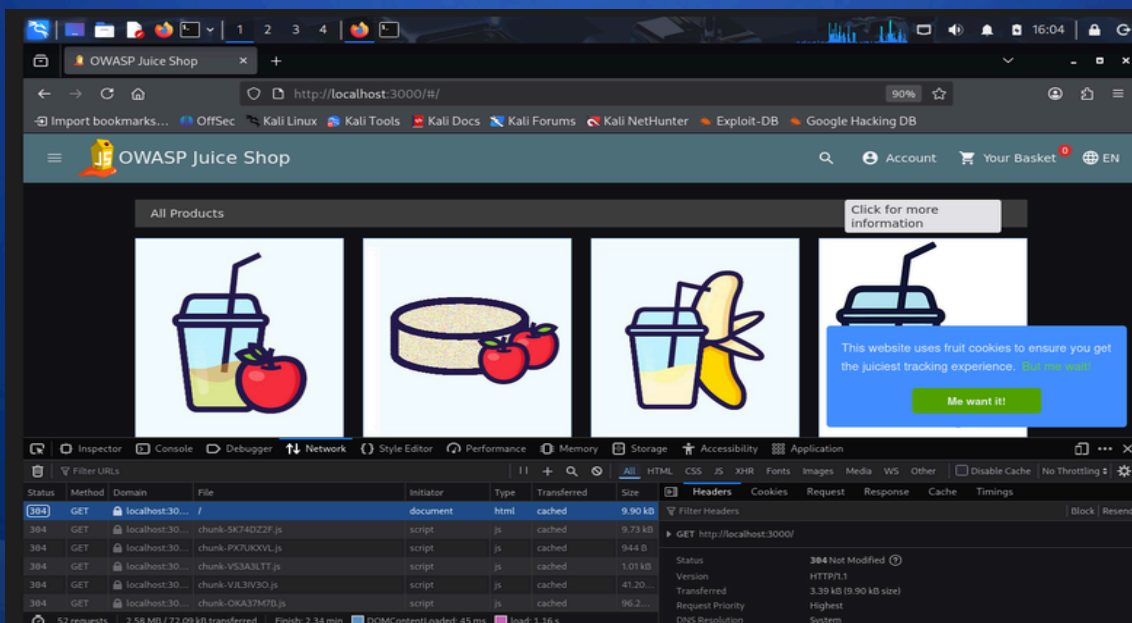


Figure 3: DevTools Network tab — all requests captured during Juice Shop page load

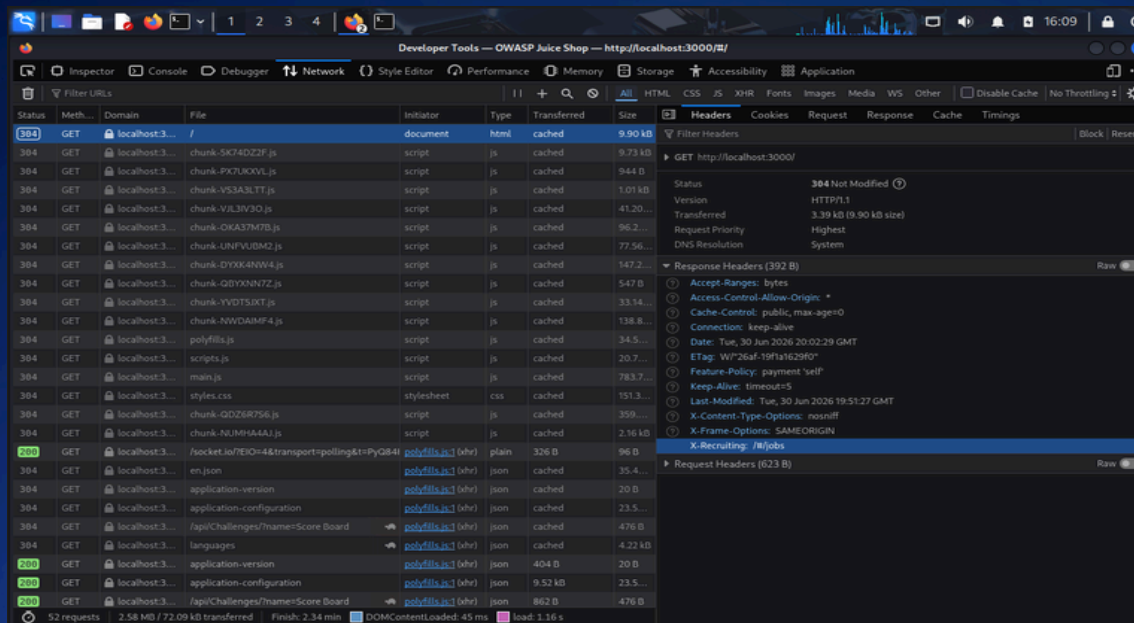


Figure 4: Response Headers — Access-Control-Allow-Origin: * and X-Recruiting: /#/jobs visible; CSP and HSTS absent

3.3 Passive Vulnerability Scan — OWASP ZAP

OWASPZAP2.17.0was configured asaFirefoxproxyfor passive traffic capture. No active scanning was performed. ZAP identified 2 alert types across 5 instances.

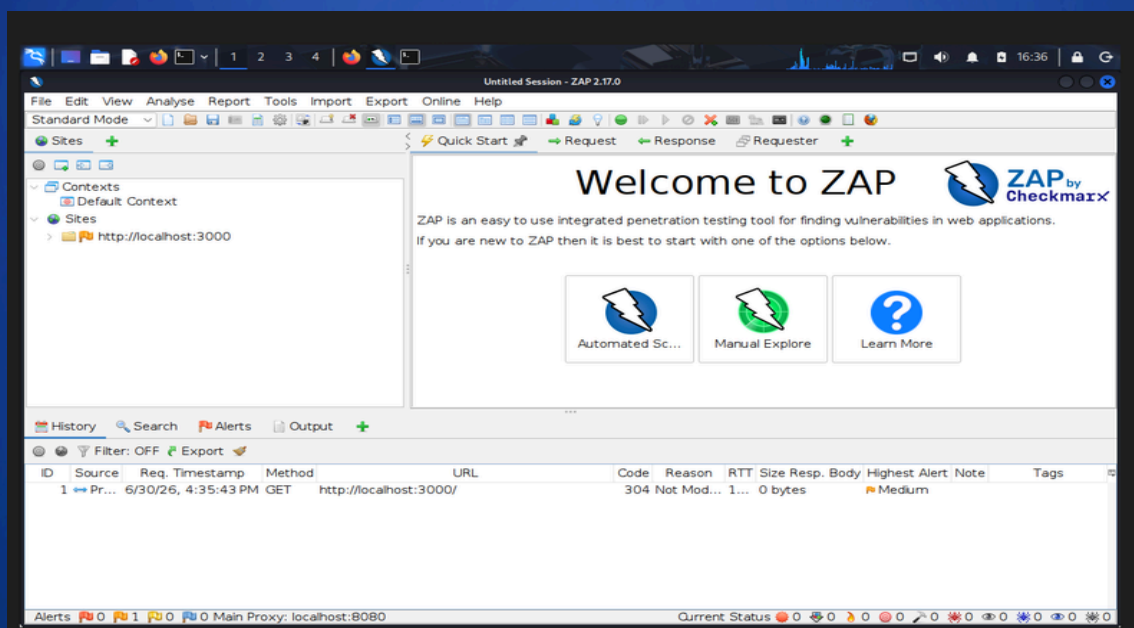


Figure 5: ZAP History — captured traffic with Medium alert on first request

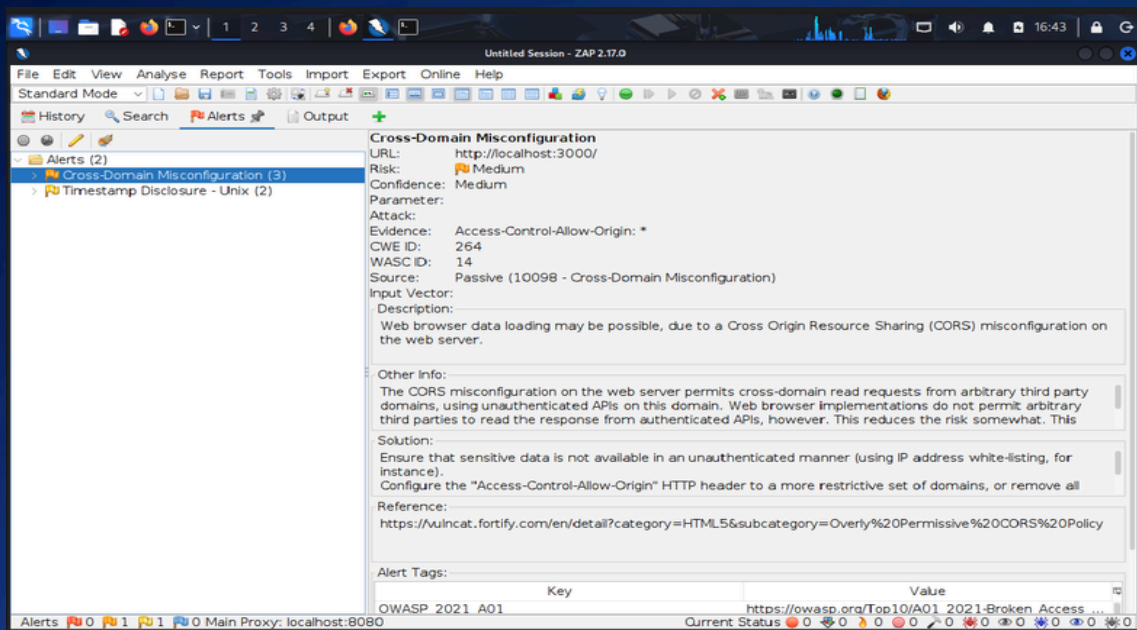


Figure 6: ZAP Cross-Domain Misconfiguration — Evidence: Access-Control-Allow-Origin: * (Medium Risk, CWE-264)

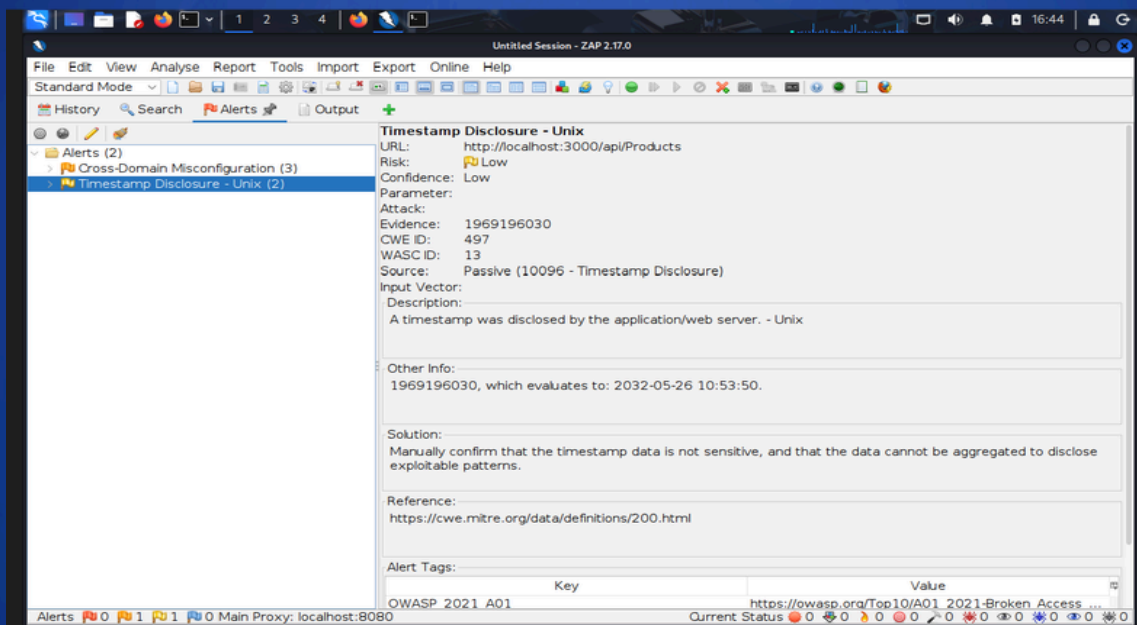


Figure 7: ZAP Timestamp Disclosure — Unix timestamp exposed in /api/Products (Low Risk, CWE-497)

4. Vulnerability Findings & Remediation

F-01 — Wildcard CORS Policy (Access-Control-Allow-Origin: *)		MEDIUM
Tool(s)	Nmap, Browser DevTools, OWASP ZAP	
Description	The application sets Access-Control-Allow-Origin to * (wildcard), allowing any website to make cross-origin requests and read responses. Confirmed independently by all three tools.	
Business Impact	A malicious website could silently make requests on behalf of a logged-in user, stealing account data, order history, or authentication tokens.	
Remediation	Replace * with a specific trusted domain: Access-Control-Allow-Origin: https://trusted-domain.com. Never use wildcards on authenticated endpoints.	

F-02 — Missing Content-Security-Policy (CSP) Header		MEDIUM
Tool(s)	Browser DevTools	
Description	No Content-Security-Policy header is present in HTTP responses. CSP controls which resources browsers are allowed to load.	
Business Impact	Without CSP, attackers exploiting XSS vulnerabilities can inject and execute malicious scripts, stealing credentials or session tokens.	
Remediation	Add: Content-Security-Policy: default-src self and expand trusted sources as needed.	

F-03 — Missing Strict-Transport-Security (HSTS) Header		MEDIUM
Tool(s)	Browser DevTools	
Description	The Strict-Transport-Security header is absent, leaving the application vulnerable to protocol downgrade attacks.	
Business Impact	Attackers on the same network can downgrade HTTPS to plain HTTP (man-in-the-middle), exposing all data in transit.	
Remediation	Add: Strict-Transport-Security: max-age=31536000; includeSubDomains; preload to all HTTPS responses.	

F-04 — Application Served Over HTTP — No HTTPS Enforcement		MEDIUM
Tool(s)	Browser DevTools	
Description	The application runs on plain HTTP without HTTPS redirection. All data is transmitted unencrypted.	
Business Impact	All data including credentials, cookies, and personal information is visible in plain text to anyone monitoring the network.	

Remediation	Configure the server to redirect all HTTP to HTTPS. Install a valid SSL/TLS certificate and enforce HTTPS on all endpoints.
--------------------	---

F-05 — Information Disclosure via X-Recruiting Header		LOW
Tool(s)	Nmap, Browser DevTools	
Description	The server exposes a non-standard X-Recruiting: /#/jobs header in all responses, revealing an internal application path.	
Business Impact	Exposes application structure to attackers. Custom headers can reveal technology stack details aiding targeted reconnaissance.	
Remediation	Remove all non-standard HTTP response headers from production. Only include headers essential for functionality.	

F-06 — Unix Timestamp Disclosure in API Responses		LOW
Tool(s)	OWASP ZAP	
Description	ZAP detected raw Unix timestamps in /api/Products responses. Value 1969196030 (= 2032-05-26) exposes server-side timing data.	
Business Impact	Exposed timestamps reveal internal system timing patterns and can assist attackers in planning timing-based attacks.	
Remediation	Remove or sanitize raw timestamp fields from API responses. Use relative time formats instead of Unix epoch values.	

5. Findings Summary

ID	Finding	Risk	Tool Source
F-01	Wildcard CORS Policy (Access-Control-Allow-Origin: *)	MEDIUM	Nmap, Browser DevTools, OWASP ZAP
F-02	Missing Content-Security-Policy (CSP) Header	MEDIUM	Browser DevTools
F-03	Missing Strict-Transport-Security (HSTS) Header	MEDIUM	Browser DevTools
F-04	Application Served Over HTTP — No HTTPS Enforcement	MEDIUM	Browser DevTools
F-05	Information Disclosure via X-Recruiting Header	LOW	Nmap, Browser DevTools
F-06	Unix Timestamp Disclosure in API Responses	LOW	OWASP ZAP

6. Conclusion & Recommendations

The passive vulnerability assessment identified 6 security findings — 4 Medium and 2 Low severity. No High or Critical vulnerabilities were found through passive analysis alone, though active testing would likely reveal additional issues given the intentionally vulnerable nature of the application.

Priority	Finding	Effort
1 — Immediate	Enforce HTTPS across all endpoints	Medium
2 — High	Implement Content-Security-Policy header	Low
3 — High	Fix CORS wildcard (Access-Control-Allow-Origin: *)	Low
4 — Medium	Add Strict-Transport-Security (HSTS) header	Low
5 — Low	Remove X-Recruiting information disclosure header	Very Low
6 — Low	Sanitize timestamp data in API responses	Low

Addressing these findings will significantly improve the security posture against XSS, CSRF, and man-in-the-middle attacks. A follow-up assessment is recommended after remediation to verify all findings have been resolved effectively.